

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №2

по дисциплине «Логика и основы алгоритмизации в инженерных задачах»

на тему «Оценка времени выполнения программ»

Выполнили ст. гр. 25BVB1:

Димов Н.А.

Бардычев А.А.

Приняли:

к.т.н., доцент Юрова О. В.

ст. преподаватель Деев М. В.

Пенза 2026

1. Цель работы

Изучение практических методов измерения времени выполнения фрагментов кода и программ в целом с использованием функций библиотеки `<time.h>` (`<ctime>`), расчет и оценка асимптотической сложности алгоритмов (О-символика), а также экспериментальный анализ производительности алгоритма перемножения квадратных матриц и алгоритмов сортировки (сортировка Шелла, быстрая сортировка `qs` Хоара и системная функция `qsort`).

2. Лабораторное задание

Задание 1

- Вычислить порядок временной сложности алгоритма умножения квадратных матриц (О-символику).
- Оценить время выполнения программы и блока перемножения матриц с помощью функций библиотеки `<time.h>` для квадратных матриц размерами $N \in \{100, 200, 400, 1000, 2000, 4000, 10000\}$.
- Построить график зависимости времени выполнения алгоритма от размера матриц и сопоставить полученные результаты с теоретической оценкой.

Задание 2

Собрать метрики времени работы алгоритма сортировки Шелла (`shell`), быстрой сортировки (`qs`) и системной функции `qsort()` на различных наборах данных ($N = 10000$) при 4 типах начальной упорядоченности:

- Случайный набор чисел;
- Возрастающая последовательность элементов;
- Убывающая последовательность элементов;
- Пилообразная последовательность (первая половина — по возрастанию, вторая — по убыванию).

3. Теоретический анализ алгоритмов

3.1. Умножение матриц

Рассматриваемый алгоритм выполняет перемножение двух квадратных матриц A и B размером $N \times N$. Результат записывается в матрицу C . Вычисление каждого элемента $c[i][j]$ сводится к скалярному произведению i -й строки матрицы A и j -го столбца матрицы B :

$$c[i][j] = a[i][0] \cdot b[0][j] + a[i][1] \cdot b[1][j] + \dots + a[i][N-1] \cdot b[N-1][j].$$

Алгоритм содержит три вложенных цикла:

- внешний цикл по строкам i — N итераций;
- цикл по столбцам j — N итераций для каждой строки;
- внутренний цикл по индексу r — N итераций для каждой пары (i, j) .

Следовательно, общее число операций умножения и сложения имеет порядок:

$$N \cdot N \cdot N = N^3.$$

Таким образом, теоретическая временная сложность алгоритма составляет:

$$O(N^3).$$

Из этой оценки следует, что при увеличении размера матрицы в 2 раза время выполнения должно возрасть примерно в $2^3 = 8$ раз.

3.2. Алгоритмы сортировки

- **Сортировка Шелла (shell).** Представляет собой модификацию сортировки вставками, в которой используется фиксированная последовательность зазоров: $\{9, 5, 3, 2, 1\}$. На почти упорядоченных массивах алгоритм выполняет минимальное число перестановок и работает достаточно быстро, вплоть до $O(N)$. Однако на обратно упорядоченных данных из-за ограниченного набора шагов сложность может ухудшаться до $O(N^2)$.
- **Быстрая сортировка Хоара (qs).** Является рекурсивным алгоритмом. В качестве опорного элемента выбирается центральный элемент подмассива: $\text{items}[(\text{left} + \text{right}) / 2]$. В среднем случае сложность составляет $O(N \log N)$. Выбор центрального элемента обеспечивает достаточно равномерное разбиение и высокую скорость работы как на возрастающих, так и на убывающих массивах.
- **Стандартная функция `qsort()`.** Это библиотечная реализация сортировки из `<stdlib.h>`. Она использует оптимизированный вариант быстрой сортировки с защитой от худших случаев и обеспечивает устойчивую сложность порядка $O(N \log N)$.

4. Исходный код программы

В соответствии с заданием полный листинг программы в отчёт не включается. Исходный код оформлен как консольное приложение в проекте Visual Studio (файл task1.cpp). Программа написана на языке C и содержит:

- функцию shell() — реализацию сортировки Шелла;
- функцию qs() — реализацию быстрой сортировки Хоара;
- функцию task1() — перемножение квадратных матриц с динамическим выделением памяти для размеров до $N = 10000$;
- функцию task2() — измерение времени работы алгоритмов сортировки.

5. Результаты работы программы

5.1. Задание 1: Зависимость времени умножения матриц от размера N

Экспериментальные замеры времени умножения квадратных матриц приведены в таблице 1. Замер для матрицы $N = 10000$ был успешно выполнен в полном объеме.

Таблица 1 – Зависимость времени умножения матриц от размера N

Размер матрицы (N×N)	Число операций (N³)	Время умножения t1 (с)	Общее время работы t2 (с)	Теоретические значения
100	1×10^6	0.002	0.002	0.002
200	8×10^6	0.014	0.014	0.016
400	64×10^6	0.115	0.115	0.128
1000	1×10^9	2.064	2.064	2.000
2000	8×10^9	20.665	20.665	16.000
4000	6.4×10^{10}	231.061	231.061	128.000
10000	1×10^{12}	4507.915	4507.915	2000.000

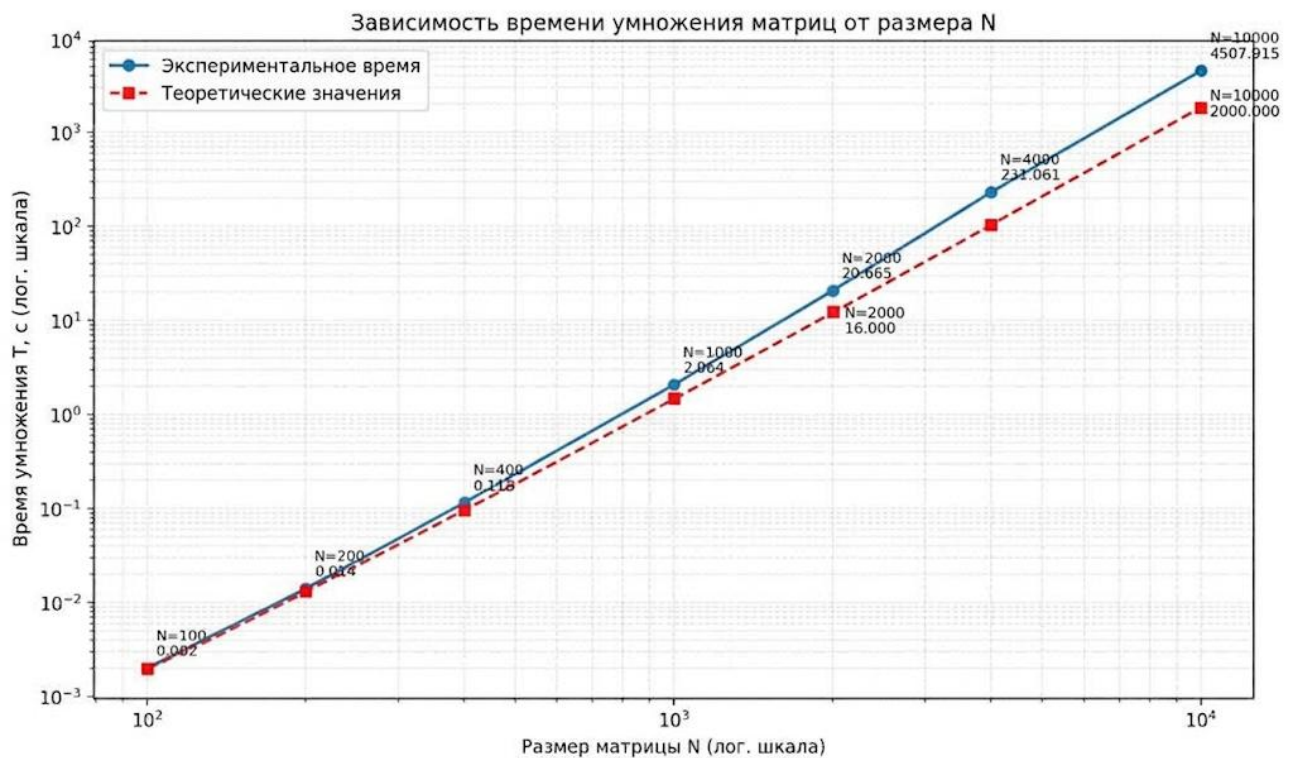


Рисунок 1 – График зависимости времени умножения матриц T от размера N

Анализ результатов Задания 1:

1. На интервале размеров N от 100 до 2000 экспериментальные замеры в целом согласуются с теоретической кубической сложностью $O(N^3)$. При удвоении размера матрицы время вычислений возрастает примерно в 8–10 раз: например, переходы $0,002 \rightarrow 0,014 \rightarrow 0,115$ с дают увеличение в 7 и 8,2 раза соответственно, а переход $1000 \rightarrow 2000 (2,064 \rightarrow 20,665 \text{ с})$ показывает рост примерно в 10 раз. Это подтверждает теоретический закон $2^3 = 8$ с учетом накладных расходов.
2. При дальнейшем увеличении размерности до $N = 4000$ и $N = 10000$ наблюдается более выраженное замедление относительно теоретической оценки. Время выполнения для $N = 10000$ составило 4307,915 с (около 71,8 мин), что превышает строго кубический рост. Это обусловлено эффектами иерархии памяти: при больших N матрицы перестают помещаться в быстрый кэш процессора. Поскольку во внутреннем цикле обход второй матрицы идет по столбцам, это вызывает дополнительные промахи кэша (cache misses) и увеличивает время доступа к оперативной памяти.

5.2. Задание 2: Сравнительный анализ алгоритмов сортировки

В таблицах 2 и 3 приведены результаты замеров времени работы алгоритмов сортировки для массива размером $N = 50000$ и $N=10000$ элементов соответственно при различных типах начальной упорядоченности.

Таблица 2 – Время выполнения алгоритмов сортировки для $N = 50000$ (в секундах)

Тип входной последовательности	Сортировка Шелла (shell)	Быстрая сортировка (qs)	Системная qsort()
1. Случайный массив	0.198 с	0.004 с	0.015 с
2. Возрастающий массив	0.001 с	0.002 с	0.022 с
3. Убывающий массив	0.39 с	0.002 с	0.02 с
4. Пилообразный массив (1/2 возр., 1/2 убыв.)	0.258 с	0.097 с	0.039 с

Таблица 3 – Время выполнения алгоритмов сортировки для $N = 10000$ (в секундах)

Тип входной последовательности	Сортировка Шелла (shell)	Быстрая сортировка (qs)	Системная qsort()
1. Случайный массив	0.008 с	0.001 с	0.003 с
2. Возрастающий массив	0.001 с	0 с (<0.001с)	0.003 с
3. Убывающий массив	0.017 с	0 с (<0.001с)	0.003 с
4. Пилообразный массив (1/2 возр., 1/2 убыв.)	0 с (<0.001с)	0 с (<0.001с)	0.003 с

Анализ результатов Задания 2:

В таблицах 2 и 3 приведены результаты замеров времени работы трёх алгоритмов сортировки на четырёх типах входных последовательностей при $N = 50000$ и $N = 10000$ соответственно.

Быстрая сортировка Хоара (qs) показала наилучшие результаты на обоих размерах массива. При $N = 10000$ на возрастающем, убывающем и пилообразном массивах время работы оказалось ниже разрешающей способности таймера (0 с (<0.001с)), а на случайном составило всего 0.001 с. При увеличении размера до $N = 50000$ время выросло незначительно: до 0.002–0.004 с на монотонных массивах и до 0.097 с на пилообразной последовательности. Такой характер роста хорошо согласуется с теоретической

оценкой $O(N \log N)$: при увеличении N в 5 раз время возрастает примерно в 6–7 раз, а не в 25. Небольшое увеличение времени на пилообразном массиве объясняется менее равномерным разбиением при выборе центрального элемента в качестве опорного.

Системная функция `qsort()` продемонстрировала высокую устойчивость к типу входных данных. При $N = 10000$ её время составило стабильные 0.003 с на всех четырёх наборах. При $N = 50000$ наблюдался небольшой разброс: от 0.015 с на случайном массиве до 0.022 с и 0.039 с на возрастающем и пилообразном соответственно. Функция уступает специализированной `qs` в 3–7 раз из-за накладных расходов на вызов функции-компаратора через указатель и обобщённый интерфейс, но остаётся надёжным инструментом с предсказуемым временем работы.

Сортировка Шелла (`shell`) оказалась наиболее чувствительной к исходному порядку элементов, причём эта зависимость ярко проявляется на обоих размерах массива:

- Возрастающий массив. Время минимально: 0.001 с при $N = 10000$ и 0.001 с при $N = 50000$. Это соответствует теоретической оценке $O(N)$ для почти отсортированных данных.

- Пилообразный массив. При $N = 10000$ время ниже разрешения таймера (0 с (<0.001 с)), при $N = 50000$ — 0.258 с. Резкий рост объясняется тем, что вторая половина массива упорядочена по убыванию и требует большого числа перестановок.

- Случайный массив. Время растёт с 0.008 с при $N = 10000$ до 0.198 с при $N = 50000$, то есть примерно в 25 раз при увеличении (N в 5 раз. Это близко к квадратичному закону $O(N^2)$.

- Убывающий массив. Наблюдается наихудший результат: 0.017 с при $N = 10000$ и 0.390 с при $N = 50000$. Рост времени более чем в 20 раз при увеличении размера в 5 раз подтверждает деградацию сложности к $O(N^2)$ из-за фиксированного набора малых шагов `gap = {9, 5, 3, 2, 1}`.

6. Выводы

- Освоены практические методы измерения времени выполнения программ на языке C с использованием функций библиотеки `<time.h>` (`clock()`, `CLOCKS_PER_SEC`).

- Экспериментально подтверждена теоретическая кубическая сложность $O(N^3)$ алгоритма перемножения матриц. На диапазоне $N \leq 1000$ экспериментальное время хорошо согласуется с теоретической оценкой $t_{\text{теор}} = C * N^3$.

Для матрицы $N = 10000$ время выполнения составило 4507,915 с (около 75 минут), что более чем в 2 раза превышает теоретическую оценку. Это объясняется промахами кэш-памяти и неидеальной локальностью данных при обходе второй матрицы по столбцам.

- Проведено сравнительное тестирование трёх алгоритмов сортировки (Шелла, qs Хоара, qsort) на 4 типах начальной упорядоченности данных при двух размерах массива: $N = 10000$ и $N = 50000$.
- Быстрая сортировка qs подтвердила свою высокую эффективность на обоих размерах. При $N = 10000$ на упорядоченных массивах время работы оказалось ниже разрешающей способности таймера, при $N = 50000$ составило 0.002–0.004 с на монотонных последовательностях и 0.097 с на пилообразной. Рост времени при увеличении N в 5 раз оказался близок к теоретическому $O(N \log N)$.
- Стандартная функция qsort() показала стабильное время работы на всех типах данных: 0.003 с при $N = 10000$ и 0.015–0.039 с при $N = 50000$. Она уступает специализированной qs в 3–7 раз из-за накладных расходов на компаратор и обобщённый интерфейс, но остаётся надёжным и универсальным инструментом.
- Сортировка Шелла подтвердила сильную зависимость от структуры данных. На возрастающем массиве она работает крайне быстро (0.001 с при обоих размерах), однако на убывающем массиве время резко возрастает: с 0.017 с при $N = 10000$ до 0.390 с при $N = 50000$. При увеличении размера в 5 раз время выросло более чем в 20 раз, что соответствует квадратичной сложности $O(N^2)$.
- Сравнение $N = 10000$ и $N = 50000$ наглядно подтверждает теоретические оценки:
 - алгоритмы с логарифмическим множителем (qs, qsort) демонстрируют умеренный рост времени;
 - сортировка Шелла на неблагоприятных данных показывает рост, близкий к N^2 , что делает её непригодной для больших массивов со случайной или обратной упорядоченностью.
- Полученные результаты согласуются с теоретическими оценками сложности: $O(N^3)$ для умножения матриц, $O(N \log N)$ в среднем для быстрой сортировки и qsort(), $O(N)$ – $O(N^2)$ для сортировки Шелла в зависимости от исходной упорядоченности данных.